

SIMATS Engineering

SIMATS Online Programming Lab — Python Programming (Case Study)

Course Name	Python Programming
Course Code	CSA0804
Name	Monish.L
Register Number	1972260035
Topic	CO4-AT2-CASE STUDY
Question	Q1 — Personal Finance Management Package in Python (50 Marks)
Section	Section E — Reporting & Analytics (10 Marks)

Q1. Section E — Reporting & Analytics [10 Marks]: Generate and verify the monthly report, budget-vs-actual analysis, and spending analytics for the Personal Finance Management package.

reports.py computes five analytics over the July 2026 transaction set loaded in Section C: a monthly income/expense/savings summary, an expense breakdown by category, budget-vs-actual status per category, the overall savings rate, and the top spending categories. All figures below are taken directly from the actual generated monthly_report.txt.

reports.py

```
"""
reports.py
-----
Reporting & analytics layer: monthly summaries, category
breakdowns, budget-vs-actual, savings rate, and top spending
categories. Writes formatted text reports to disk.
"""

from collections import defaultdict

def monthly_summary(manager, year, month):
    txns = manager.transactions_in_month(year, month)
    income = round(sum(t.amount for t in txns if t.txn_type == "Income"), 2)
    expense = round(sum(t.amount for t in txns if t.txn_type == "Expense"), 2)
    net = round(income - expense, 2)
    return {"year": year, "month": month, "income": income, "expense": expense,
            "net_savings": net, "transaction_count": len(txns)}

def category_breakdown(manager, year, month, txn_type="Expense"):
    txns = manager.transactions_in_month(year, month)
    totals = defaultdict(float)
    for t in txns:
        if t.txn_type == txn_type:
            totals[t.category] += t.amount
    return {cat: round(amt, 2) for cat, amt in sorted(totals.items(), key=lambda x: -x[1])}

def budget_vs_actual(manager, budget_manager, year, month):
    spend = category_breakdown(manager, year, month, txn_type="Expense")
    all_categories = set(budget_manager.budgets) | set(spend)
    rows = []
    for category in sorted(all_categories):
        spent = spend.get(category, 0.0)
```

```

        rows.append(budget_manager.check(category, spent))
    return rows

def savings_rate(manager, year, month):
    summary = monthly_summary(manager, year, month)
    if summary["income"] == 0:
        return 0.0
    return round(summary["net_savings"] / summary["income"] * 100, 2)

def top_categories(manager, year, month, n=3, txn_type="Expense"):
    breakdown = category_breakdown(manager, year, month, txn_type)
    return list(breakdown.items())[:n]

def write_monthly_report(manager, budget_manager, year, month, out_path="monthly_report.txt"):
    summary = monthly_summary(manager, year, month)
    breakdown = category_breakdown(manager, year, month)
    budget_rows = budget_vs_actual(manager, budget_manager, year, month)
    rate = savings_rate(manager, year, month)
    top3 = top_categories(manager, year, month, 3)

    with open(out_path, "w") as f:
        f.write("PERSONAL FINANCE MANAGEMENT - MONTHLY REPORT\n")
        f.write(f"Period: {year}-{month:02d}\n")
        f.write("-" * 55 + "\n\n")

        f.write("SUMMARY\n")
        f.write("-" * 55 + "\n")
        f.write(f"Total Income:           Rs.{summary['income']:.2f}\n")
        f.write(f"Total Expense:           Rs.{summary['expense']:.2f}\n")
        f.write(f"Net Savings:             Rs.{summary['net_savings']:.2f}\n")
        f.write(f"Savings Rate:            {rate:.2f}%\n")
        f.write(f"Transactions Logged: {summary['transaction_count']}\n\n")

        f.write("EXPENSE BREAKDOWN BY CATEGORY\n")
        f.write("-" * 55 + "\n")
        for cat, amt in breakdown.items():
            f.write(f"{cat:<20}Rs.{amt:.2f}\n")
        f.write("\n")

        f.write("BUDGET VS ACTUAL\n")
        f.write("-" * 55 + "\n")
        f.write(f"{'Category':<20}{'Budget':<12}{'Spent':<12}{'Status':<15}\n")
        for row in budget_rows:
            limit_str = f"Rs.{row['limit']:.2f}" if row["limit"] is not None else "N/A"
            f.write(f"{row['category']:<20}{limit_str:<12}Rs.{row['spent']:<9.2f}{row['status']:<15}\n")
        f.write("\n")

        f.write("TOP SPENDING CATEGORIES\n")
        f.write("-" * 55 + "\n")
        for i, (cat, amt) in enumerate(top3, start=1):
            f.write(f"{i}. {cat}: Rs.{amt:.2f}\n")

    return {
        "summary": summary, "breakdown": breakdown,
        "budget_rows": budget_rows, "savings_rate": rate, "top3": top3,
    }

```

Actual generated monthly report txt

```

PERSONAL FINANCE MANAGEMENT - MONTHLY REPORT
Period: 2026-07
=====

SUMMARY
-----

```

Total Income: Rs.65500.00
Total Expense: Rs.38700.00
Net Savings: Rs.26800.00
Savings Rate: 40.92%
Transactions Logged: 18

EXPENSE BREAKDOWN BY CATEGORY

Rent	Rs.15000.00
Food	Rs.6700.00
Shopping	Rs.5500.00
Utilities	Rs.3800.00
Entertainment	Rs.2700.00
Transport	Rs.2000.00
Education	Rs.1800.00
Healthcare	Rs.1200.00

BUDGET VS ACTUAL

Category	Budget	Spent	Status
Education	Rs.2000.00	Rs.1800.00	Within Budget
Entertainment	Rs.2500.00	Rs.2700.00	Exceeded
Food	Rs.6000.00	Rs.6700.00	Exceeded
Healthcare	Rs.3000.00	Rs.1200.00	Within Budget
Rent	Rs.15000.00	Rs.15000.00	Within Budget
Shopping	Rs.4000.00	Rs.5500.00	Exceeded
Transport	Rs.3000.00	Rs.2000.00	Within Budget
Utilities	Rs.4000.00	Rs.3800.00	Within Budget

TOP SPENDING CATEGORIES

1. Rent: Rs.15000.00
2. Food: Rs.6700.00
3. Shopping: Rs.5500.00

Analysis

1. Monthly summary

Total income for July 2026 is Rs.65,500.00 (Salary Rs.55,000 + Freelance Rs.8,000 + Investment Rs.2,500) and total expense is Rs.38,700.00 across 15 expense transactions, giving net savings of Rs.26,800.00 across 18 logged transactions in total — matching the manual check $65500 - 38700 = 26800$.

2. Expense breakdown by category

Rent is the single largest expense at Rs.15,000.00, followed by Food at Rs.6,700.00 (three separate grocery/dining transactions summed) and Shopping at Rs.5,500.00. Utilities, Entertainment, Transport, Education, and Healthcare make up the remainder, and the eight listed categories sum to exactly the total expense figure:

$15000 + 6700 + 5500 + 3800 + 2700 + 2000 + 1800 + 1200 = 38700$, confirming the breakdown is internally consistent with the summary.

3. Budget vs actual

Three of the eight budgeted categories were exceeded this month: Food (Rs.6,700 spent against a Rs.6,000 limit, over by Rs.700), Entertainment (Rs.2,700 against Rs.2,500, over by Rs.200), and Shopping (Rs.5,500 against Rs.4,000, over by Rs.1,500). The remaining five categories — Rent, Utilities, Transport, Healthcare, and Education — stayed within their configured limits. This is exactly the kind of actionable signal the Section A requirements called for: a user scanning this table immediately sees Shopping as the biggest budget overrun in absolute terms.

4. Savings rate

$\text{Savings rate} = \text{net savings} \div \text{income} \times 100 = 26800 \div 65500 \times 100 = 40.92\%$, matching the report exactly. A savings rate above 30–40% is generally considered healthy, so this figure by itself already tells the user their month was

financially sound despite three categories running over budget — the overruns were absorbed by income rather than eroding savings.

5. Top spending categories

The `top_categories()` function confirms the three largest expense categories in order: 1. Rent (Rs.15,000.00), 2. Food (Rs.6,700.00), 3. Shopping (Rs.5,500.00) — together accounting for Rs.27,200.00, or about 70% of all expense for the month ($27200 / 38700 \times 100 \approx 70.3\%$), showing that a small number of categories dominate total spend, which is typical for personal budgets and is exactly the kind of concentration a top-N report is meant to surface.

Verification against Section C's raw numbers

Metric	Manually Computed	Reported	Match
Total Income	Rs.65,500.00	Rs.65,500.00	YES
Total Expense	Rs.38,700.00	Rs.38,700.00	YES
Net Savings	Rs.26,800.00	Rs.26,800.00	YES
Savings Rate	40.92%	40.92%	YES

Conclusion

Every figure in the generated `monthly_report.txt` was independently recomputed by hand from the raw `transactions_input.csv` values and matched exactly, confirming the reporting layer is correct. Beyond raw correctness, the report answers the questions a real user would ask: how much did I save (Rs.26,800.00, a 40.92% savings rate), which categories went over budget (Food, Entertainment, Shopping), and where is most of my money going (Rent, Food, and Shopping account for roughly 70% of expense) — delivering on the analytics requirement (FR4) defined back in Section A.